Gomoku

Inledning

Detta dokument är en sammanfattning av hur vi i grupp 2 har jobbat med vårat projekt och refererar till vårt github repossitory som finns här: https://github.com/SebTns/Gomoku.git I slutet så finns reflektioner som vi i gruppen funnit i arbetet och efter det så kommer några personliga som jag själv funnit på vägen.

Kravfångst

Vi började med ett samtal med kunden. Det enda kunden gav oss var en user story: **”Som spelare vill jag kunna spela ett parti Gomoku, så att jag kan vinna, förlora eller spela oavgjort.”** Kunden sa ingenting om hur systemet skulle byggas, bara vilket resultat hen förväntade sig.

Kravfångst är arbetet med att ta reda på vad som måste gälla för att det resultatet ska inträffa, och att formulera det så exakt att det går att testa. Vår definition: **ett krav är en förväntan som är testbar. Går den inte att testa är det inget krav.**

Det viktiga är hur vi frågar. Vi frågar inte ”vilka krav har du?”, för kunden ska inte behöva kunna ord som funktionellt krav, API eller databas. I stället ställde vi följdfrågor:

- **Vilken webbläsare?** Det spelar ingen roll för kunden. Alltså ska spelet fungera i alla vanliga webbläsare
- **Vilken enhet?** Både en mobil-telefon och en laptop. Alltså krävs stöd för både mobil, surfplatta och dator. Oberoende av vilket operativsystem de har installerat (IOs, Android, Windows, Linux med flera).
- **Mot vem spelar du?** Oftast ensam mot datorn, men ibland med en kompis. Alltså behövs både spel mot AI och spel mot en vän.
- **Vart ska partiet spela?** Det kan vara online eller på samma enhet (hot Seat).
- **Behöver man ha ett konto?** Nej det ska gå att spela utan att registrera ett konto.
- **Är hastigheten på appen viktigt?** Ja, om den är för långsam så kommer det att bli jobbigt att spela och kunden kommer att slut med det.
- **Internet uppkopplingen är den avgörande?** Ja, ibland händer det att kundens internet strular vilket leder till att appen behöver kunna komma ihåg partiet tills det är färdigspelat.

Detta är några av svaren vi fått under vår intervju. Samtliga svar vi får fram hjälper oss att formulera följande vitala delar för att få fram en produkt med hög kvalitet.

- användningsområden (UC)
- funktionella krav (FR)
- icke-funktionella krav (NFR)
- acceptanskriterier (AC).

Begreppslista

Det räcker inte att hitta kraven. Vi måste också mena samma sak när vi använder samma ord. Exempel: ”fem i rad” betydde exakt fem för en av oss och fem eller fler för en annan. Så länge ett ord har två betydelser går det inte att testa, och då är det inte längre ett krav.

Därför har vi en begreppslista (00-begreppslista.md), vårt gemensamma språk. Den har tre delar: domäntermer, GDPR-termer och tekniska termer. Den fastställer bland annat:

- **5 i rad**: exakt fem stenar i samma färg i följd. Det är det enda vinstvillkoret.
- **6 eller mer i rad**: räknas inte som vinst (överlinje).
- **Sten**: permanent när den väl har placerats.
- **Svart** gör alltid första draget.
- **Oavgjort**: brädet är helt fullt och ingen har fem i rad.

Den viktigaste regeln: **vid konflikt gäller begreppslistan.**

Begreppsmodell

Med ett gemensamt språk kan vi beskriva systemet som en modell (05-begreppsmodell.md). Modellen är statisk. Den beskriver vad systemet består av, inte vad det gör. Centrala begrepp är Parti, Spelare (med underklasserna Mänsklig Spelare och AI Motståndare), Spelbord, Korsning, Sten, Drag, Fem I Rad och Statistik.

Modellen slår fast tre regler:

1. **Exakt fem.** Fem I Rad består av exakt fem stenar (Sten [5]), vilket motsvarar FR-08.14 och FR-08.15.
2. **Två spelare, en abstraktion.** Ett parti har exakt två spelare. Både människa och AI ärver från Spelare, så spellogiken behöver inte veta vem motståndaren är. Det gör det möjligt att uppfylla NFR-09.1, som kräver att spellogiken är skild från gränssnittet.
3. **Stenen är permanent.** Att ångra ett drag (UC-19) är därför en operation på Drag, inte på Sten.

Diagram

När vi har fångat upp en massa information av kunden och läst på om hur Gomoku fungerar och vilka regler spelat har så börjar vi rita upp diagram som visar grafiskt hur vår app ska fungera.
Det finns olika typer av diagram som man kan ta hjälp av.

- **Aktivitetsdiagram** är en typ av flödesschema som visar flödet från en aktivitet eller åtgärd till en annan inom system eller en process
- **Sekvensdiagram visar** hur olika objekt eller system delar kommunicerar med varandra i en exakt tidsföljd.
- **Tillståndsdiagram** visar vilka olika tillstånd ett objekt eller system kan befinna sig i samt vad som får det att byta tillstånd.
- Use case-diagram visar vilka funktioner ett system ska ha och vem som ska använda dem.

Vi har exempel på diagram i vårt projekt under ”docks/diagrams/uml”. Dock så har vi vår use-case - diagram i filen 07-use-cases-overview.md. När vi gjort dessa så blir det lättare och tydligare hur vi ska jobba vidare med våra krav.

Use cases och krav

Diagrammen visar hur systemet ska se ut och hur vi kan testa det. Vi skapar Use Cases utifrån det funktioner som kommit fram från intervjun och diagrammen. Use-casen beskriver flöden som en aktörkan göra. I dessa UC så får man en referens till vilka krav som behöver uppfyllas för att få ett lyckat resultat. Kraven delas in i tre typer:

Typ	Exempel från presentationen
Funktionella krav (FR)	FR-08.14 Exakt fem i rad ger vinst. FR-08.15 Sex eller fler i rad ger inte vinst.
Icke-funktionella krav (NFR)	NFR-02 Prestanda och svarstid, NFR-07 Säkerhet och dataskydd, NFR-13 Språk i gränssnittet
Kompletterande krav (SR)	SR-01.3 Exakt fem i rad (affärsregel), SR-02 Tekniska begränsningar (React-SPA, REST över HTTPS, WebSockets, PostgreSQL, Docker), SR-03.1 GDPR gäller

Vi utgår från kundens user story om vinst och följer den genom kedjan. En och samma regel, exakt fem i rad, återkommer i begreppslistan, som affärsregel (SR-01.3), som funktionella krav (FR-08.14/15), i begreppsmodellen och i acceptanskriterierna.

GDPR som konsekvens av use cases

Eftersom systemet behandlar personuppgifter om EU-invånare omfattas det av GDPR (SR-03.1). Kravet leder i sin tur till nya use cases, som var och en ger egna funktionella och icke-funktionella krav:

Use case	Funktionella krav	Icke-funktionella krav
UC-NFR-01 Ge samtycke till cookie	FR-24	NFR-07.4, NFR-07.5
UC-NFR-03 Informeras om delning med tredje part	FR-27, FR-31	NFR-10, NFR-11
UC-NFR-04 Begär radering av data	FR-30	NFR-12 (GDPR artikel 17)

Även UC-26 Spara parti påverkas, eftersom sparad speldata kan vara personuppgifter.

Artikel 17, ”rätten att bli bortglömd”, ger den registrerade rätt att få sina personuppgifter raderade utan onödigt dröjsmål. Hos oss blir det bland annat följande krav:

- En verifierad raderingsbegäran ska vara genomförd inom 30 dagar (NFR-12.2, SR-03.2).
- Spelaren får en bekräftelse via e-post (NFR-12.3).
- Raderade uppgifter får inte kunna återskapas från säkerhetskopior, loggar, cacheminnen eller exporter (NFR-12.4).
- Speldata som sparas ska vara anonymiserad (NFR-12.5).
- Samtyckesregister sparas anonymiserade i minst 3 år (NFR-12.6).

AC och ACT – hur vi kontrollerar att kraven är uppfyllda

- **AC (Acceptance Criteria)**: vad som måste vara uppfyllt.
- **ACT (Acceptance Test)**: hur vi kontrollerar att det är uppfyllt.
- **Spårbarhet**: krav-ID ↔ use case ↔ test.

Acceptanskriterierna skrivs i formatet **Given / When / Then**. Exempel, AC-22-01 Logga in:

Steg	Beskrivning
Given	Spelaren har ett konto och är på inloggningssidan.
When	Spelaren skriver rätt e-postadress och lösenord.
Then	Spelaren loggas in och kommer till huvudmenyn. (I repot ingår även att överföringen sker med TLS 1.2 eller senare.)

Hur vet vi att projektet är klart?

Vi går tillbaka till kundens ursprungliga förväntan: **”Jag som användare vill kunna spela ett parti av Gomoku där jag kan vinna, förlora eller spela oavgjort.”**

Att vi själva tycker att spelet känns bra räcker inte. Kunden godkänner leveransen när:

- **Acceptanskriterierna** är uppfyllda (enligt vår bedömning).
- **Acceptanstesterna** är godkända av kunden.

När båda är uppfyllda kan vi lämna över till kunden.

Personliga krav att analysera

Fyra användningsfall behandlas individuellt. Urvalet jag gjort för att täcka olika aktörer, olika sorters krav och — viktigast — **olika svåra testproblem**. Fyra varianter av samma lätta testfall hade inte sagt något.

§	Use Case	Aktör	Testproblemet det illustrerar
1	UC-02 Gör ett drag	Spelare	Ett testfall som skiljer rätt från nästan rätt
2	UC-28 Rapportera ett tekniskt problem	Spelare → Administratör	Testförutsättning utanför applikationen
3	UC-29 Tillfälligt blockera en spelare	Administratör	Tidsgränser i tre storleksordningar
4	UC-NFR-04 Begär radering av data	Registrerad spelare, DPO	Ett krav som inte går att bevisa

Varje testfall avslutas med ett aktivitetsdiagram över **testaktiviteten** — inte över användningsfallet. Diagrammen visar hur kontrollen utförs och var den kan gå sönder.

---

1. Gör ett drag

Use Case

Fält	Värde
ID	UC-02
Aktör	Spelare (primär), Motståndare och AI-motståndare (sekundära)
Förutsättningar	Ett parti med status `PÅGÅENDE` finns. Det är spelarens tur. Brädet visas.
Krav	FR-08.1 – FR-08.17 · NFR-02.2, NFR-02.4, NFR-06.3 – NFR-06.5, NFR-08.2

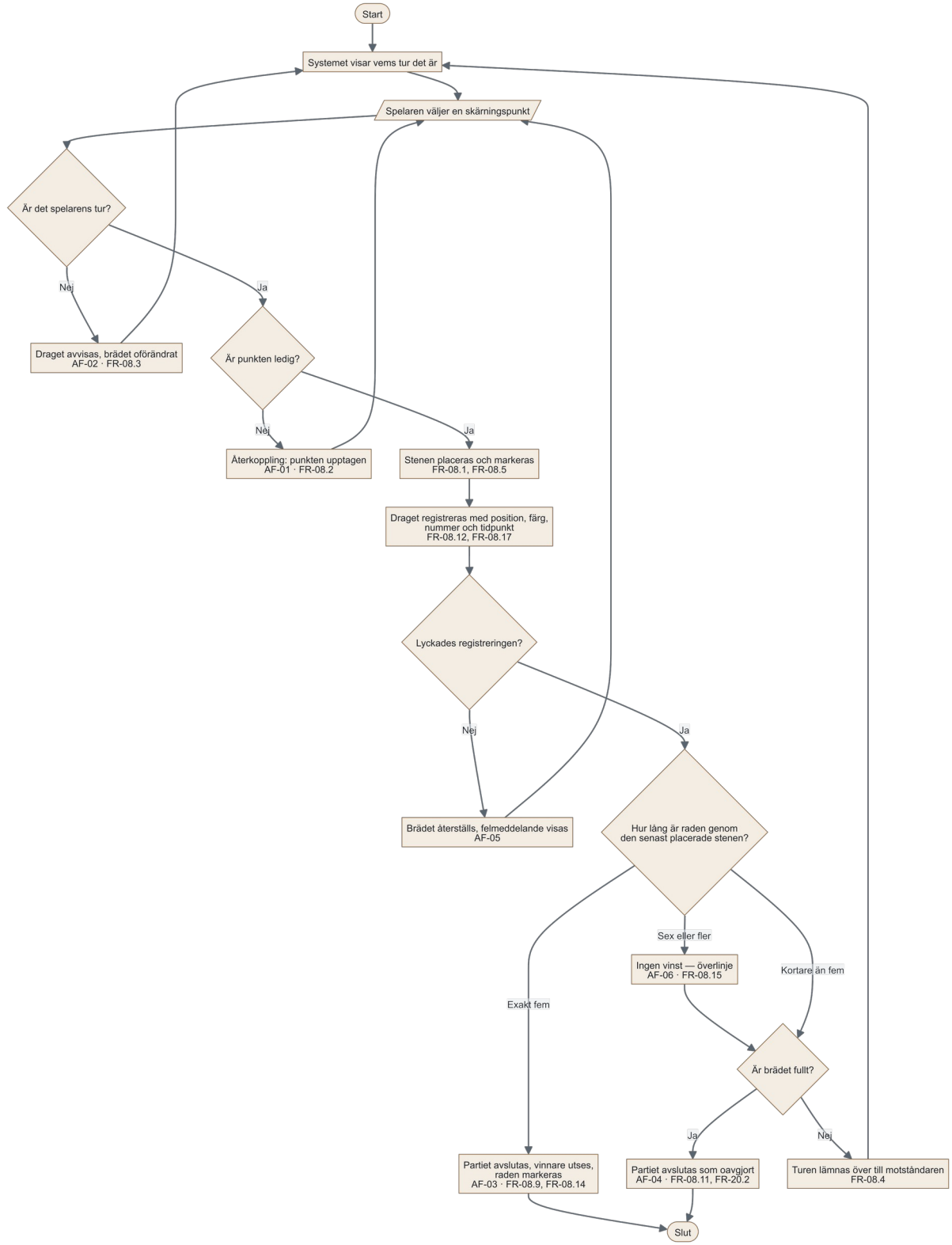
Aktivitet (huvudflöde): Systemet visar vems tur det är. Spelaren väljer en ledig skärningspunkt. Systemet kontrollerar att punkten är ledig, placerar stenen, markerar den, ökar dragräknaren och registrerar draget med position, färg, dragnummer och tidpunkt. Systemet kontrollerar om den senast placerade stenen ingår i en obruten rad av exakt fem. Om inte, och om brädet inte är fullt, lämnas turen över.

Alternativa flöden: AF-01 punkten upptagen · AF-02 utanför sin tur · AF-03 exakt fem i rad · AF-04 brädet fullt · AF-05 registreringen misslyckas · AF-06 överlinje, sex eller fler.

Postcondition, lyckat: Draget är registrerat, brädet uppdaterat och turen överlämnad — eller partiet avslutat med ett resultat.

Postcondition, misslyckat: Brädet är oförändrat, ingen tur har bytts och spelaren har fått besked om varför.

Hela flödet med samtliga avvikelser:



Test Case

Det förväntade resultatet verifieras av nio acceptanskriterier, AC-02-01 till AC-02-09. Åtta av dem är rutin. **AC-02-06 är det som är värt att uppehålla sig vid**, eftersom det är det enda som skiljer en korrekt implementation från en som är nästan korrekt.

Kriterium	Verifierar	Typ
AC-02-01	Sten placeras, registreras, återkoppling inom 100 ms	Positivt

Kriterium	Verifierar	Typ
AC-02-02	Turen lämnas över	Positivt
AC-02-03	Upptagen punkt avvisas, turen kvar	Negativt
AC-02-04	Drag utanför sin tur avvisas	Negativt
AC-02-05	Exakt fem ger vinst inom 100 ms	Positivt
AC-02-06	**Sex eller fler ger inte vinst**	**Negativt**
AC-02-06b	**Lucka fylld till exakt fem ger vinst**	**Positivt**
AC-02-07	Fullt bräde ger oavgjort	Positivt
AC-02-08	Misslyckad registrering återställer brädet	Negativt
AC-02-09	Draget går att göra med tangentbord	Positivt

AC-02-06 i BDT-form:

> ***Given*** *att ett parti pågår och det är svarts tur*
> ***And*** *svart har tre stenar i rad, en ledig punkt, och därefter tre svarta stenar på samma linje*
> ***And*** *ingen av grupperna är fem i rad, så partiet har status PÅGÅENDE*
> ***When*** *svart placerar en sten på den lediga punkten mellan grupperna*
> ***And*** *raden därmed blir sju stenar lång*
> ***Then*** *ska systemet inte utse någon vinnare*
> ***And*** *partiet ska fortsätta med status PÅGÅENDE*
> ***And*** *turen ska lämnas över till vit*

När kan en överlinje alls uppstå? Bara när ett drag **fyller en lucka**. En rad som byggs en sten i taget kommer aldrig förbi fem, eftersom partiet avgörs i samma ögonblick raden blir fem lång — fem i rad är en vinst oavsett om ändarna är blockerade. Blockerade ändar spelar roll för hotbilden i spelet, inte för vinstvillkoret.

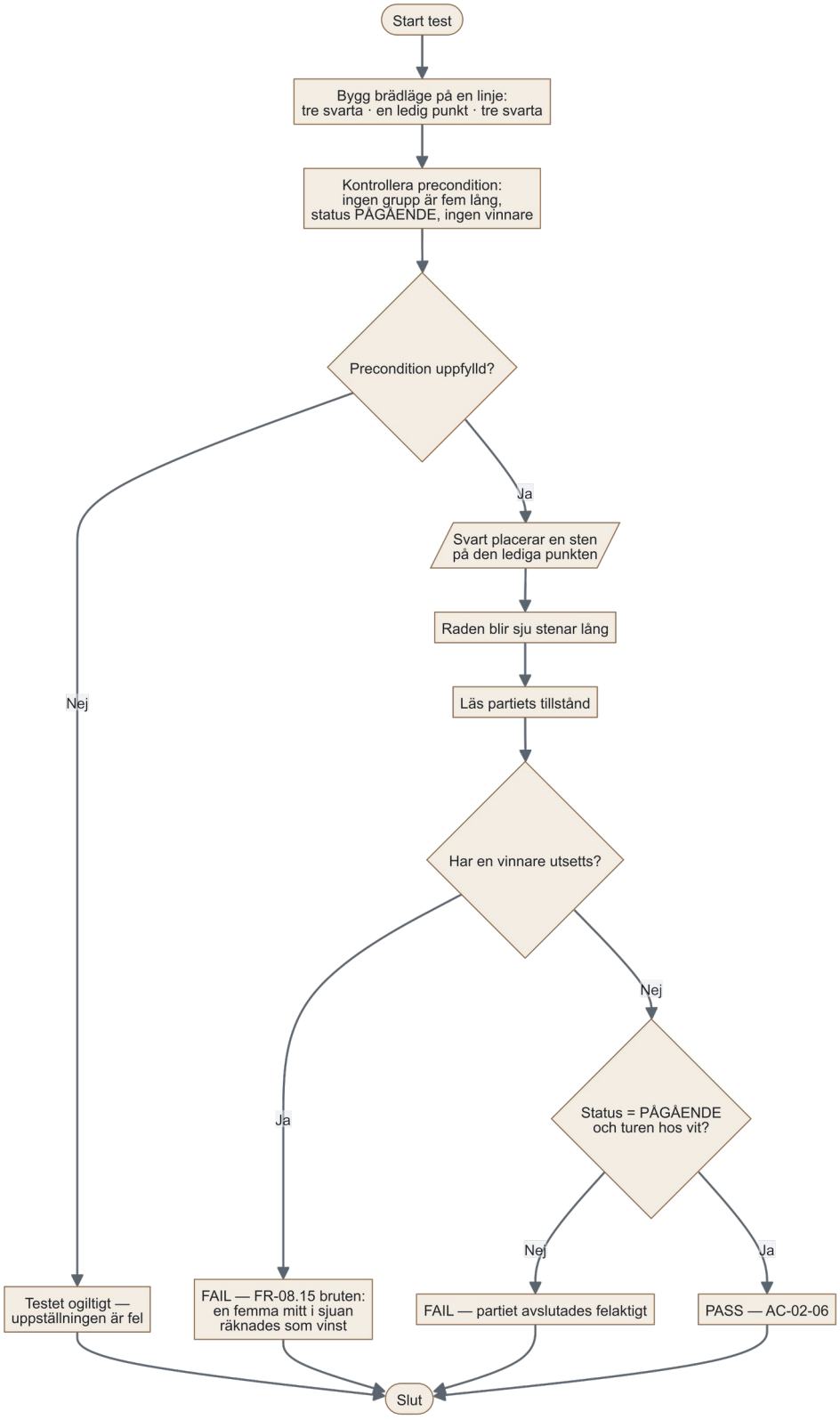
Det brädläge som krävs är alltså två grupper av samma färg på samma linje med exakt en ledig punkt emellan, där ingen grupp är fem lång. Giltiga kombinationer är 1+4, 2+3, 2+4, 3+3, 3+4 och 4+4, vilket efter draget ger sex till nio i rad.

Varför just detta kriterium. En implementation som söker efter fem intilliggande stenar av samma färg hittar en träff mitt i sjuan och utropar vinst. Den är fel enligt FR-08.15 — men den passerar varje test som lägger stenar en i taget, eftersom ett sådant test aldrig kan skapa en överlinje. Skillnaden syns bara med ett *konstruerat* brädläge.

Det är också därför FR-08.16 finns: den placerade stenen hamnar **mitt i** raden, inte i änden, så kontrollen måste utgå från den och mäta åt båda hållen. En implementation som bara räknar framåt ser tre stenar och missar hela sjuan.

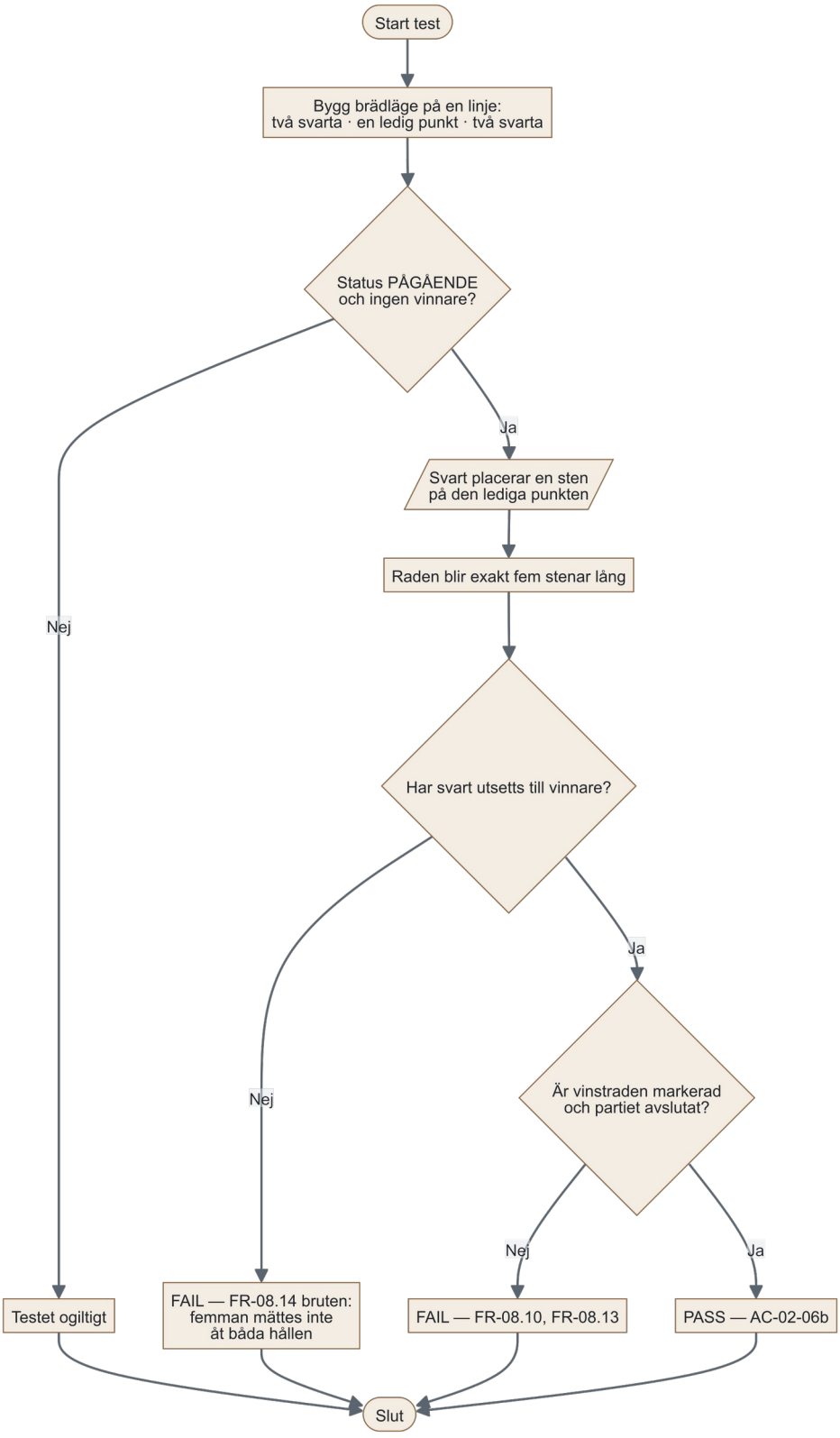
Testaktiviteten inleds med en förutsättningskontroll av just detta: om någon av grupperna redan är fem lång är partiet avgjort innan draget, och testet mäter något annat än det avser.

Aktivitetsdiagram för testaktiviteten:



Det komplementära kriteriet. AC-02-06b har samma sorts brädläge — två grupper med en lucka emellan — men motsatt förväntat resultat. Två svarta, lucka, två svarta: fylls luckan blir raden **exakt** fem och det *är* en vinst. Skillnaden mot AC-02-06 är bara radens totala längd efter draget.

Tillsammans låser de två kriterierna fast att kontrollen mäter hela raden genom den placerade stenen. Var för sig räcker de inte: AC-02-06b passerar även för en implementation som räknar fel uppåt, och AC-02-06 passerar även för en som aldrig utser någon vinnare alls.



2 Rapportera ett tekniskt problem

Use Case

Fält	Värde
ID	UC-28
Aktör	Spelare (primär), Administratör (sekundär)
Förutsättning	Spelaren har öppnat spelet. Ett tekniskt problem har uppstått och upptäckts.
Krav	FR-28.1 – FR-28.10 · NFR-03.1 – NFR-03.7, NFR-04.1, NFR-04.4 – NFR-04.6

Aktivitet: Spelaren öppnar "Report a Problem". Systemet visar formuläret och vilka uppgifter som bifogas automatiskt. Spelaren beskriver problemet och skickar. Systemet kontrollerar att en beskrivning finns, registrerar rapporten, tilldelar ett unikt rapport-ID och en tidsstämpel, och visar en bekräftelse inom 3 sekunder.

Alternativa flöden: AF-01 beskrivning saknas · AF-02 rapporten kan inte registreras, sparas lokalt · AF-03 spelaren avbryter · AF-04 rapporten skickas om från lokal lagring.

Postcondition, lyckat: Rapporten är registrerad med unikt ID och tidsstämpel. Ett eventuellt pågående parti är oförändrat.

Postcondition, misslyckat: Rapporten är inte registrerad, spelaren är informerad, och rapporten finns kvar lokalt så att den kan skickas om.

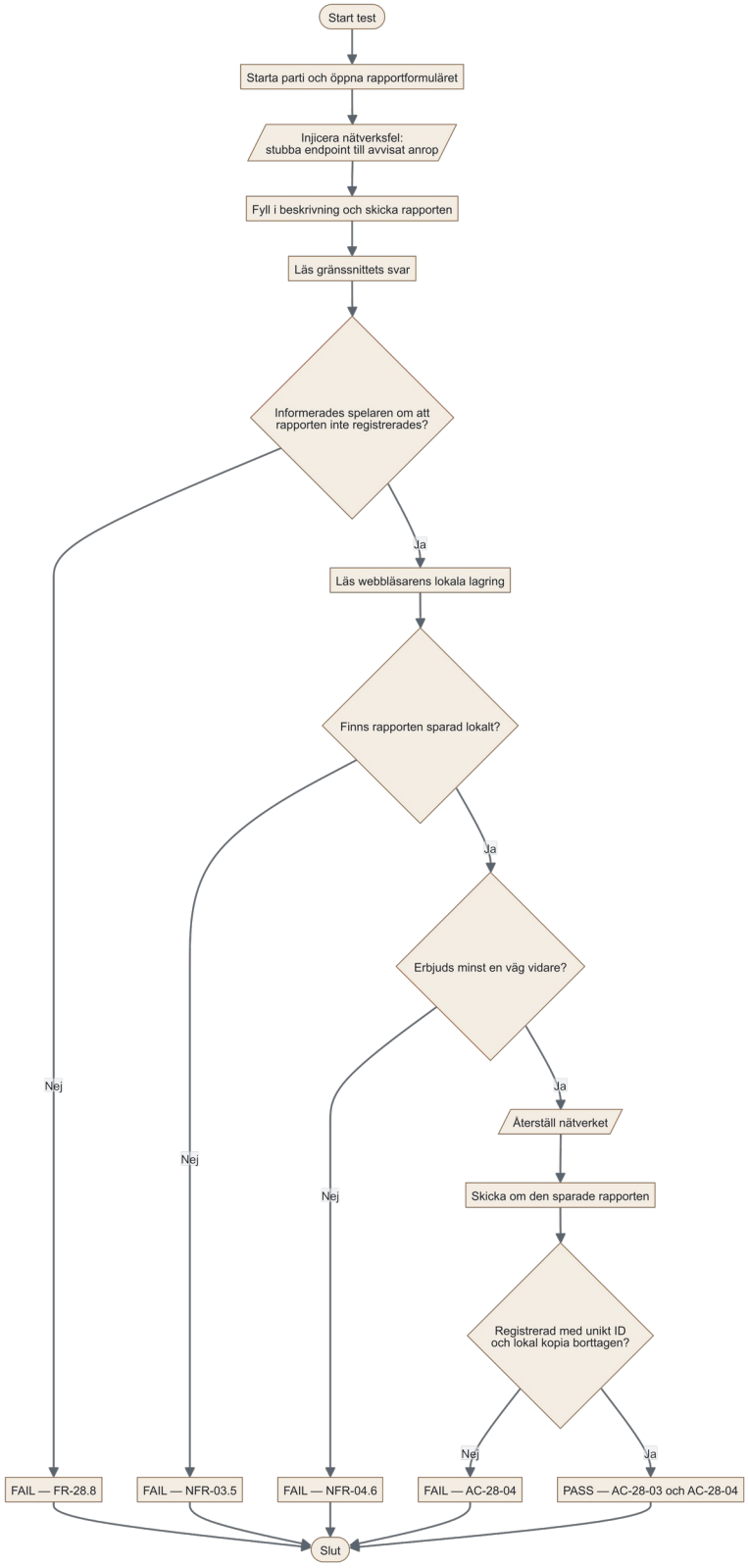
Test Case

Sju acceptanskriterier, AC-28-01 till AC-28-07. Det intressanta är **AC-28-03**, som verifierar NFR-03.5: om rapporten inte kan skickas ska den sparas lokalt.

Testproblemet. För att testa vad systemet gör när systemet inte går att nå måste felet framkallas utifrån — genom en nätverksstubb, en avstängd anslutning eller en Proxy som avvisar anropet. Testförutsättningen ligger alltså **utanför applikationen**, och kriteriet kan därför inte skrivas som ett rent enhetstest. Det är en direkt följd av att kravet handlar om systemets beteende vid sin egen otillgänglighet.

Testet blir dessutom bara meningsfullt om det fortsätter förbi felet: att rapporten sparas lokalt har inget värde om den aldrig kan skickas om. Därför sträcker sig testaktiviteten över både AC-28-03 och AC-28-04 och avslutas med att den lokala kopian ska vara borta.

Aktivitetsdiagram för testaktiviteten:



En motsägelse som testet blottade. NFR-03.3 sa ursprungligen att rapporten ska innehålla webbläsare, tidpunkt och parti-ID "utan personuppgifter utöver spelarnamnet". Formuleringen läser som att spelarnamnet inte vore en personuppgift — men `00-begreppslista.md` definierar personuppgifter som all information som rör en identifierbar person och räknar uttryckligen upp användarnamn. Begreppslistan är normerande. Kravet är uppdelat i NFR-03.3, NFR-03.6 och NFR-03.7 så att insamlingen är uttalad och minimerad i stället för bortdefinierad, och AC-28-07 testar att spelaren informeras om vad som bifogas.

3 Tillfälligt blockera en spelare

Use Case

Fält	Värde
ID	UC-29
Aktör	Administratör (primär), blockerad spelare (sekundär)
Förutsättning	Administratören har tillgång till modereringsfunktionen. Spelaren finns i systemet. Det finns ett modereringsärende.
Krav	FR-29.1 – FR-29.15 · NFR-05.1 – NFR-05.7, NFR-04.1

Aktivitet: Administratören öppnar modereringsärendet, granskar informationen, väljer att blockera, väljer längd och omfattning, och bekräftar. Systemet genomför blockeringen, registrerar spelare, start- och sluttid i UTC, omfattning, beslutsfattande administratör och ärende, samt hindrar spelaren från att påbörja nya partier. När tiden gått ut hävs blockeringen automatiskt.

Alternativa flöden: AF-01 administratören avbryter · AF-02 blockeringen kan inte genomföras · AF-03 spelaren deltar i ett pågående parti · AF-04 administratören häver i förtid.

Postcondition, lyckat: Spelaren är blockerad. Beslutet, omfattningen, giltighetstiden och beslutsfattaren är registrerade. Administratören har fått bekräftelse och spelaren är informerad.

Postcondition, misslyckat: Spelaren är inte blockerad och administratören är informerad.

Test Case

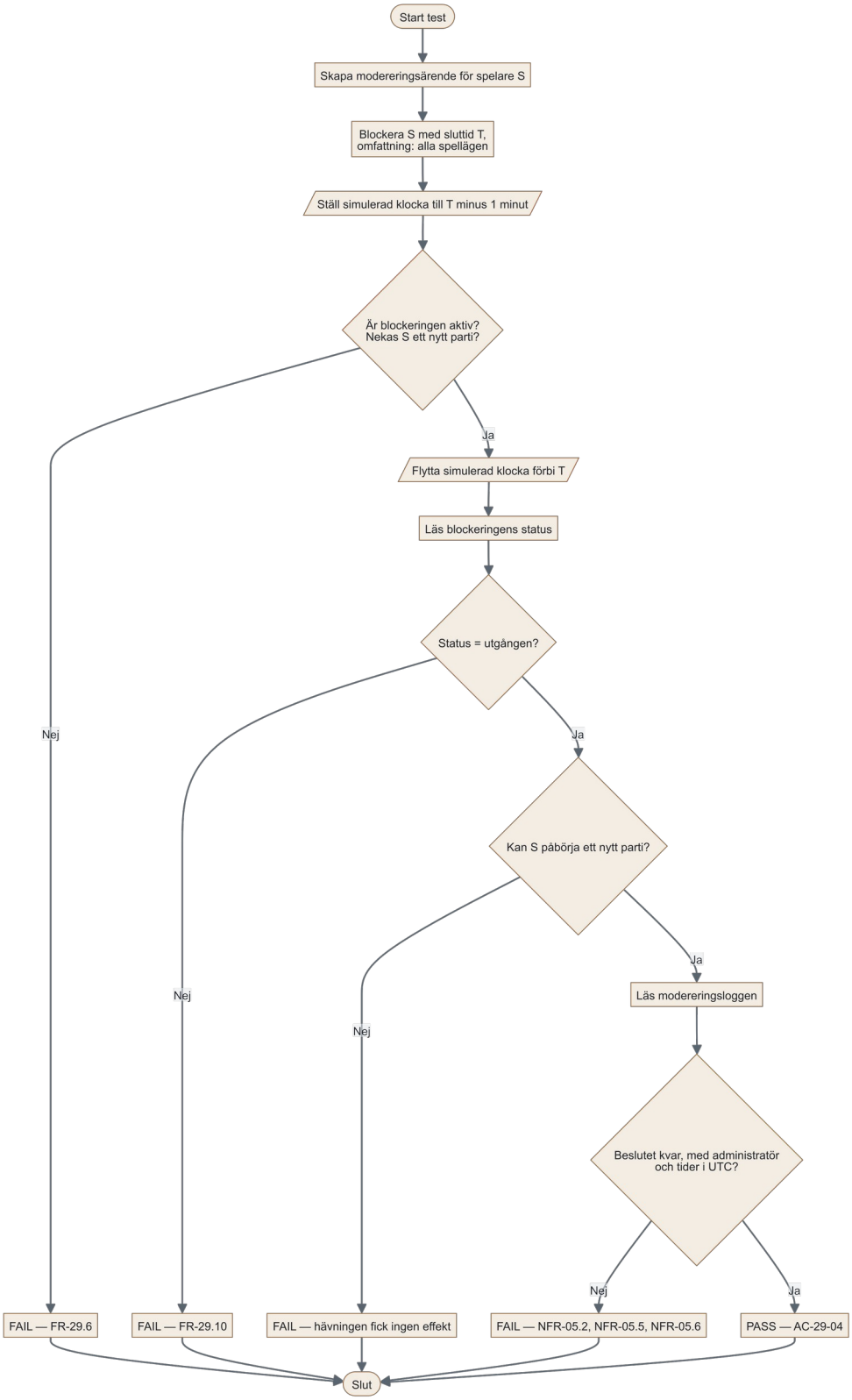
Åtta acceptanskriterier, AC-29-01 till AC-29-08. Här är **tidsgränserna** testproblemet, och de ligger i tre helt olika storleksordningar:

Krav	Gräns	Testbarhet
NFR-05.1	Blockeringen träder i kraft inom **1 minut**	Direkt mätbar i test
NFR-05.4	Rapporter hämtbara inom **24 timmar**	Mätbar med simulerad tid
NFR-05.2	Moderationsbeslut sparas i **12 månader**	Går inte att testa

Hur testar man ett krav på 12 månader? Svaret är att man inte gör det. Ingen testsvit väntar ut ett år, och ett test som gör det är värdelöst eftersom det aldrig hinner ge återkoppling. NFR-05.7 lades till för att göra problemet hanterbart: systemet ska kunna visa en blockerings status vid en godtycklig tidpunkt utan att tiden behöver löpa i realtid. Då kan AC-29-04 verifiera den automatiska hävningen med en simulerad klocka.

Men det är viktigt att vara tydlig med vad som då faktiskt verifieras. Ett test med simulerad tid visar att **logiken** är rätt — att statusen beräknas korrekt för en given tidpunkt. Det visar inte att beslutet fortfarande finns kvar om tolv verkliga månader. Den delen av NFR-05.2 är inte ett testfall utan en granskning av lagringsschemat och en kontroll av att inget automatiskt raderingsjobb tar bort posten i förtid.

Aktivitetsdiagram för testaktiviteten:



Fyra öppna frågor som blev krav. UC-29 hade fyra obesvarade frågor i filen. Att skriva testfallet gick inte utan att besvara dem, eftersom ett testfall kräver ett förväntat resultat:

Fråga	Beslut	Krav
Ska spelaren få veta att hen blockerats?	Ja, vid nästa inloggnings- eller spelstartsförsök, med sluttid	FR-29.11

Fråga	Beslut	Krav
Ska orsaken anges?	Orsakskategori ja, rapportörens identitet nej	FR-29.12
Vad händer vid pågående parti?	Partiet får spelas klart, inga nya partier	FR-29.13
Gäller blockeringen alla spellägen?	Administratören väljer omfattning per beslut	FR-29.14

Det tredje var det svåraste och saknade Eftervillkor helt. Att avbryta ett pågående parti skulle straffa motståndaren, som inte gjort något fel.

---

4 Begär radering av data

Use Case

Fält	Värde
ID	UC-NFR-04
Aktör	Registrerad spelare (primär); Administratör, DPO och E-posttjänst (sekundära)
Förutsättning	Spelaren har personuppgifter i systemet. En raderingsbegäran är skapad och verifierad.
Krav	FR-30.1 – FR-30.13 · NFR-12.1 – NFR-12.7, NFR-07.1, NFR-07.5 · SR-03.1 – SR-03.4
GDPR	Artikel 17

Aktivitet: Systemet tar emot den verifierade begäran, bekräftar mottagandet och anger senaste datum för radering. Systemet identifierar alla personuppgifter kopplade till spelaren, raderar eller anonymiserar varje kategori enligt lagringsschemat, markerar kontot som raderat, sätter status till GENOMFÖRD och skickar bekräftelse.

Alternativa flöden: AF-01 radering kan inte slutföras inom 30 dagar · AF-02 rättslig reservation hindrar fullständig radering · AF-03 begäran kan inte verifieras.

Postcondition, lyckat: Alla personuppgifter är raderade eller anonymiserade inom 30 dagar. Status är GENOMFÖRD. Bekräftelse är skickad. Samtyckes- och begärandeuppgifter finns kvar i icke-identifierbar form.

Postcondition, misslyckat: Raderingen är inte genomförd, DPO är underrättat och spelaren har informerats om orsak och nytt datum.

Flödet är systemdrivet. Till skillnad från de tre föregående börjar detta användningsfall inte med att en aktör gör något, utan med att systemet tar emot en redan verifierad begäran. Det är därför ett sekvensdiagram passar bättre än ett aktivitetsdiagram för själva flödet — e-posttjänsten och dataskyddsombudet är egna deltagare med egen livslinje.

Test Case

Nio acceptanskriterier, AC-NFR-04-01 till AC-NFR-04-09. Ett av dem går inte att uppfylla.

AC-NFR-04-05 verifierar NFR-12.4: personuppgifter ska inte kunna återskapas efter radering.

Det är ett **negativt påstående om ett tillstånd som ska vara omöjligt**, och sådana går inte att bevisa genom test. Ett test kan visa att uppgifterna *går* att återskapa från en viss sökväg — aldrig att det är omöjligt från varje tänkbar sökväg. Testet kan alltså bara falsifiera kravet, aldrig bekräfta det.

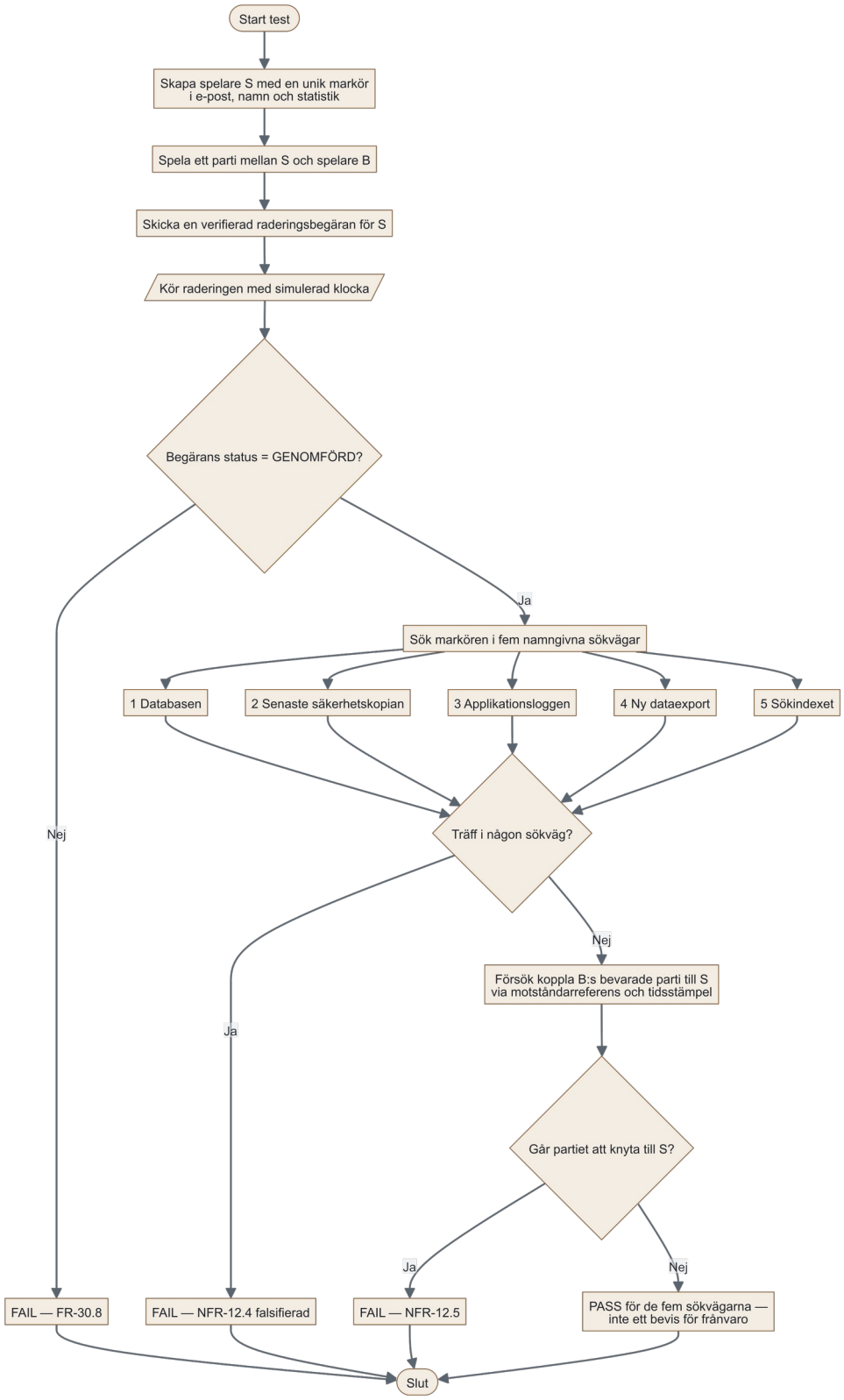
Det praktiska svaret är att skriva om kravet till en avgränsad, kontrollerbar form. Kriteriet räknar upp fem namngivna sökvägar i stället för att gömma frågan bakom orden "inte kan återställas":

1. databasen
2. den senaste säkerhetskopian
3. applikationsloggen
4. en ny dataexport
5. sökindexet

Ett godkänt test visar då att uppgifterna inte gick att hitta i fem sökvägar. Det är inte samma sak som kravet i GDPR-mening, och skillnaden är medveten. **Varje ny lagringsplats som tillkommer i systemet måste läggas till i listan**, annars slutar testet tyst att täcka kravet — vilket är den farligaste sortens testfall: ett som fortsätter lysa grönt efter att det upphört att mäta något.

Testaktiviteten fortsätter dessutom förbi de fem sökvägarna, eftersom anonymisering som bara nollställer ett spelar-ID inte håller: om motståndarens parti fortfarande går att koppla till den raderade spelaren via motståndarreferens och tidsstämpel, så är uppgiften inte anonymiserad utan pseudonymiserad. Begreppslistan skiljer uttryckligen på de två.

Aktivitetsdiagram för testaktiviteten:



30-dagarsgränsen. SR-03.2 gör kravet testbart i teorin och opraktiskt i en testsvit. NFR-12.7 finns därför: raderingen ska gå att verifiera utan att 30 dagar förflyter. Skillnaden mellan ett verifierbart krav och ett krav man faktiskt kan verifiera i en pipeline är inte akademisk — det är skillnaden mellan ett krav som kontrolleras vid varje push och ett som kontrolleras en gång och sedan glöms.

Reflektion

Vänta inte till slutet

Problemet: en funktion kraschar precis när det är dags att lämna över, och vi måste ta bort den, trots att den motsvarar ett viktigt krav. Konkret exempel: inloggningsfunktionen kraschar.

Förebyggande lösning: anpassa tidigt och samarbeta med kunden genom demos och avstämningsmöten. Vid varje möte redovisas och diskuteras de problem som har uppstått, så att prioriteringar kan göras tillsammans med kunden i stället för i sista minuten.

Vår första idé var att skriva acceptanskriterier direkt utifrån våra user stories. Det fungerade inte:

- Det blev för många kriterier.
- Olika user stories gav samma acceptanskriterier.
- Det blev onödigt komplicerat.

Vi misslyckades tidigt (”fail fast”) och ändrade arbetssätt. **Acceptanskriterierna måste kopplas till ett krav, funktionellt eller icke-funktionellt, och inte till en user story.** Den nya kedjan blev:

Krav → Task → AC (flera) → ACT

Bra kravfångst ger bra acceptanskriterier, som ger acceptanstest hos kunden, som leder till godkännande.

I repot (10-reflektioner.md) finns också konkreta motsägelser som vi hittade och löste. Två exempel:

- **Vinstvillkoret:** exakt fem eller fem eller fler? Löst med begreppslistan samt FR-08.14 och FR-08.15.
- **Svarstiden:** 0,33 sekunder eller 100 ms? Kravet NFR-02.2 gäller före acceptanskriteriet.

Våra lärdomar

Lärdom	Varför det spelar roll
Testbara krav	Tydliga mål och förutsättningar ger hög kvalitet.
Samverkan	Vi arbetar samtidigt på samma problem med olika mål, vilket ger en bättre produkt.
Spårbara krav	Vi upptäcker fel även långt in i processen och kan lösa dem.
Konkreta krav	Minskar risken för missförstånd och hjälper oss hitta motsägelser i systemet.
Testningen börjar när kravet skrivs	Vi ställer oss alltid frågan: går det här att testa?
Gemensamt språk	Begreppslistan minskar missförstånd, och hela teamet använder samma ord.

Målet är konkreta, testbara krav utan onödiga krav, och acceptanstester som kunden godkänner.

Vad vi tar med oss

Nyttan med spårbarhet: vi kan hitta fel i arbetet och förstå varför lösningen ser ut som den gör. Det ger en tryggare arbetsprocess som:

- är tydligare,
- hittar fel i arbetet,
- bygger samverkan, eftersom teammedlemmarna inte behöver tolka samma krav på olika sätt. Alla ser samma mål, samma krav och hur de ska verifieras.

Förväntat resultat → Krav → Use case → AC/ACT → Test → Godkännande → Överlämning

Vad är mina reflektioner från kursen?

Den först stora saken för mig efter att par kurser men ganska svart/vita definitioner av tester/krav och kvalitet ska vara. Men det är väldigt individuellt beroende på erfarenhet och synsätt. Jag gillar verkligen de mer generella uttrycken som vi blivit introducerade för. ”Kvalitet är när det förväntade resultatet inträffar” detta uttryck är sant för var och en av oss. Om jag kokar pasta så förväntar jag mig att den ska vara ”al dente” medan min son vill att den ska vara genom kokt. Så för att vi båda ska få kvalitet så får jag lyfta ur en del av pastan för att få den som jag vill och låta resten koka lite till tills den är genom kokt och min son också får kvalitet.
Hur vet jag om det är ett krav?
Föreställ dig att du ber mig att göra en kopp kaffe till dig.

- **Ett ogiltigt "krav" (inte testbart):**
  Du säger: *"Jag vill ha ett **gott** kaffe."*
  Detta är inte ett riktigt krav eftersom "gott" är en känsla. Om jag ger dig kaffet och du säger *"det här var inte gott"*, så kan vi inte bevisa vem som har rätt. Det går inte att testa objektivt.
- **Ett riktigt krav (testbart):**
  Du säger: *"Jag vill ha ett kaffe med **två teskedar socker och en skvätt mjölk**."*
  Detta är ett riktigt krav. När jag räcker över koppen kan vi **testa och kontrollera** om det stämmer. Vi kan titta på om mjölken är i, och vi kunde mäta upp exakt två skedar socker. Det går att svara **JA** eller **NEJ** på om kravet är uppfyllt.

Så när vi fått definitionen ”Ett krav är en förväntan som är testbar. Om den inte är testbar är det inte ett krav.” så har det gjort det mycket enklare att verkligen avgöra vad som är ett krav.

Vi satt i vår grupp och jag får frågan om hur kan man beskriva acceptanstest. Då kom jag att tänka på när vi pratade om kvalitet och man kommer ut från matbutiken med en kasse mat. För att vi ska kunna komma ut med mat så behöver butiken ha varor. Vår inköpslista blir då acceptanskriterier och vi testar dom genom att lägga maten i korgen/vagnen. Butikens acceptanstest för att vi ska få lämna butiken med maten är att vi betalar för de varor vi lagt i korgen/vagnen och får ett kvitto på att testet blivit godkänt. Så de vi upptäckte var ju att vi har våra acceptanstester och butiken har sina och bådas tester behöver bli godkända för att förväntat resultat ska uppstå.

En praktisk reflektion var när jag fick testa på att par programmera vilket verkligen gjorde att jag fick upp hastigheten på mitt skrivande och att om något blev fel så kunde det korrigeras på en gång.

En gång en programmerare alltid en programmerare, det var underbart att börja knacka kod igen TACK!

Källor finns i repot

- https://github.com/SebTns/Gomoku.git